

Aufgabenheft

Tag 10

Skalierbare digitale Prozessarchitektur – Microservices, Kubernetes, TDD/DevOps und Operating Model in elf Aufgaben.

11 Aufgaben · L1 · L2 · L3

Niveau-Mix:
3 L1 · 5 L2 · 3 L3

Bearbeitung:
Selbstbearbeitung im Lernpfad

Dozent:
Can Siebert

Begleitend:
Lehrskript & Lösungsheft

So arbeiten Sie mit diesem Heft

Dieses Aufgabenheft enthält elf Aufgaben zu Tag 10 – *Skalierbare digitale Prozessarchitektur*: Microservices, Container/Kubernetes, TDD und Teststrategien sowie DevOps-Strategien inklusive Operating Model. Die Aufgaben sind in drei Niveaus gegliedert:

- **L1 • Wissen** – **Wissen.** Begriffe, Servicegrenzen, Pipeline-Logik, Testpyramide. 5–10 Minuten pro Aufgabe.
- **L2 • Anwendung** – **Anwendung.** Servicegrenzen ziehen, Betriebskonzept entwerfen, SLOs definieren, RACI bauen. 15–30 Minuten.
- **L3 • Management** – **Management.** Operating Model, Entscheidungsarchitektur, Reliability-Framework. 30–45 Minuten.

Jede Aufgabe enthält:

- eine Niveau-Markierung und einen Zeit-Richtwert,
- den Aufgabentext (häufig mit Fallbeschreibung),
- einen Antwort-Freiraum für Ihre Lösung,
- eine *YouTube-Suchquery* für den begleitenden Lernpfad.

Hinweis

Die Musterlösungen finden Sie im separaten *Lösungsheft Tag 10*. Versuchen Sie jede Aufgabe zuerst eigenständig. Der Mehrwert entsteht im Entwerfen und Argumentieren – nicht im Abschreiben.

Niveau L1 – Wissen & Reproduktion

Hilfsvideos zu diesem Tag

Die folgenden YouTube-Erklärvideos vermitteln die Inhalte, die Sie zur Bearbeitung der Aufgaben benötigen. Klicken Sie auf den Titel, um das Video direkt zu öffnen; die vollständige URL steht in Klammern.

1. Microservices — warum, wann und mit welchen Grenzen

<https://youtu.be/ZXfLy229GLs>

(URL: `https://youtu.be/ZXfLy229GLs`)

2. Kubernetes — Container-Orchestrierung in der Praxis

<https://youtu.be/SFVg7fDdvLE>

(URL: `https://youtu.be/SFVg7fDdvLE`)

3. Test-Driven Development — Qualität als Entscheidungsinstrument

<https://youtu.be/71nLhdZuMk0>

(URL: `https://youtu.be/71nLhdZuMk0`)

4. DevOps-Operating-Model — „you build it, you run it“ und Plattform-Teams

<https://youtu.be/ak3oE0I6cXU>

(URL: `https://youtu.be/ak3oE0I6cXU`)

Tipp: Öffnen Sie das Video, lassen Sie es im Hintergrund laufen und arbeiten Sie parallel an der Aufgabe.

Aufgabe 1 – Microservices – Eignung, Risiken und Servicegrenze

L1 • Wissen

~ 8 min

Erläutern Sie in eigenen Worten, was *Microservices* sind und worin sich ihr Vorteil gegenüber einem Monolithen begründet. Beantworten Sie konkret:

1. Nennen Sie **drei Nutzenversprechen** von Microservices (z. B. unabhängige Deployments, gezielte Skalierung, klare Verantwortungsgrenzen).
2. Nennen Sie **drei Risiken**, die mit Microservices typischerweise einhergehen (z. B. Komplexität, verteilte Fehler, Observability-Bedarf).
3. Erläutern Sie in einem Satz, was eine *gute Servicegrenze* ausmacht – und warum sie *nicht* entlang von Datenbanktabellen, sondern entlang von *Geschäftsfähigkeiten (Capabilities)* verlaufen sollte.

Lernvideo: Microservices — warum, wann und mit welchen Grenzen

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Aufgabe 2 – Container & Kubernetes – Kernbegriffe

L1 • Wissen

~ 8 min

Ordnen Sie die folgenden Kernbegriffe der Container- und Kubernetes-Welt jeweils ihrer korrekten Bedeutung zu. Schreiben Sie zu jedem Begriff einen kurzen, eigenen Erklärungssatz – so, dass eine fachfremde Kollegin den Begriff versteht.

Begriffe:

1. **Container**
2. **Orchestrierung**
3. **Self-Healing**
4. **Rollback**
5. **Service-Discovery**
6. **Blue/Green-Deployment**
7. **Canary-Release**
8. **Ressourcenlimits (Requests/Limits)**

Markieren Sie zusätzlich zwei Begriffe, deren *Auslassen* im Betrieb am schnellsten zu Schaden führt – und begründen Sie kurz, warum.

Lernvideo: [Kubernetes — Container-Orchestrierung in der Praxis](#)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Aufgabe 3 – Testpyramide & Quality Gates

L1 • Wissen

~ 10 min

Erläutern Sie das Prinzip der *Testpyramide* und erstellen Sie eine **Vergleichstabelle** für die drei Teststufen *Unit*, *Integration* und *End-to-End (E2E)*.

Eigenschaft	Unit-Test	Integrationstest	E2E-Test
Was wird getestet?			
Geschwindigkeit			
Kosten / Fragilität			
Anteil in Pyramide			
Typischer Fehlerfund			

Ergänzen Sie unter der Tabelle in zwei Sätzen: Was sind *Quality Gates*, und welche drei Prüfungen würden Sie als nicht verhandelbares Gate vor einem Produktiv-Release verlangen?

Lernvideo: [Microservices — warum, wann und mit welchen Grenzen](#)

.....

.....

.....

.....

.....

.....

.....

.....

Niveau L2 – Anwendung & Transfer

Aufgabe 4 – Service-Schnitt aus Prozessbeschreibung – Order-to-Cash

L2 • Anwendung

~ 25 min

Ausgangslage: Order-to-Cash

Prozess: Bestellung → Verfügbarkeit prüfen → Preis/Angebot → Auftrag bestätigen → Versand → Rechnung → Zahlung → Reklamation (ggf.) → Reporting.

Ausgangszustand: Heute ein Monolith, Releases alle 6 Wochen, viele Abhängigkeiten zwischen Modulen.

Cheat Sheet: Service = klarer Zweck + klarer Owner + Schnittstelle (Input/Output). Gute Grenze = möglichst wenig “Hin und Her” zwischen Services (geringe Kopplung).

Arbeitsauftrag:

1. Schneiden Sie den Prozess in **5–7 Service-Kandidaten** (z. B. *Order, Inventory, Shipping, Billing, Payments, Complaints*).
2. Für **drei** dieser Services definieren Sie einen *Service-Steckbrief* mit:
 - *Zweck* (1 Satz),
 - *Input* (vier Felder) und *Output* (vier Felder),
 - *Owner-Rolle*,
 - *kritischster Fehlerfall* (1 Satz).
3. Entscheiden Sie unter Unsicherheit: Welche *eine* Service-Grenze ist am riskantesten (weil Daten- oder Ownership-Verhältnisse unklar sind)? Definieren Sie einen **Guardrail**, der das Risiko begrenzt (z. B. Anti-Corruption-Layer, fachliche Eskalation, gemeinsamer Datenvertrag).

Lernvideo: Microservices — warum, wann und mit welchen Grenzen

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Aufgabe 5 – Release-Flow & Testpyramide als Sicherheitsnetz

L2 • Anwendung

~ 25 min

Mini-Szenario

Ziel: Wöchentliches Release. **Sorge:** Ausfälle in der Produktion. **Pflicht:** Der Kernprozess *Rechnung / Zahlung* muss auditierbar bleiben.

Cheat Sheet: Deployment braucht Rollout-Plan, Rollback, Monitoring/Alerts und Verantwortliche. Tests gliedern sich in Unit (schnell), Integration (Schnittstellen), E2E (Geschäftsfluss).

Arbeitsauftrag:

1. Entwerfen Sie einen **minimalen Release-Flow** in maximal *sieben Schritten* von “Änderung” bis “Prod” inklusive zweier **Quality Gates**.
2. Skizzieren Sie eine **Testpyramide für drei Services** (z. B. Order, Billing, Payments). Geben Sie für jeden Service grob an, wie viele Tests welcher Art Sie ansetzen.
3. Definieren Sie **zwei Betriebskennzahlen** (SLO-orientiert), z. B. *Verfügbarkeit, Fehlerquote, Latenz, MTTR* – jeweils mit *Owner* und *Zielwert*.
4. Entscheidung unter Unsicherheit: Wo akzeptieren Sie im **MVP** bewusst Rest-Risiko (z. B. weniger E2E-Tests)? Wie kompensieren Sie es (Canary, Feature Toggle, manuelle Kontrolle)? Dokumentieren Sie das in einem kurzen *Risikoplan*.

Lernvideo: [Kubernetes — Container-Orchestrierung in der Praxis](#)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Aufgabe 6 – Fallstudie “InsureFast” – Servicegrenzen & Betriebskonzept

L2 • Anwendung

~ 30 min

Fallstudie: InsureFast

Unternehmen: “InsureFast” – Versicherer mit digitalem Abschluss und Schadenmeldung.

Problem: Release-Zyklus 6–8 Wochen; häufige Produktionsprobleme; der Kundenprozess *Schaden melden* bricht bei Lastspitzen; Audit fordert Nachvollziehbarkeit (*wer hat wann was entschieden?*).

Restriktionen / Unsicherheit: 12 Wochen bis Saison-Peak, Budget 220 k €, Legacy-DB, Security-Anforderungen, unklare Lastprofile, Zielkonflikt *Speed vs. Stabilität*.

Arbeitsauftrag (Teil 1 von 2):

- Servicegrenzen (5–8):** Definieren Sie Servicegrenzen entlang der Prozesskette *Schaden melden* → *prüfen* → *entscheiden* → *auszahlen* (z. B. *Claim Intake, Policy/Customer, Fraud Check, Claim Assessment, Payout, Notifications, Document Management*). Begründen Sie eine Grenze, die nicht trivial ist.
- Kubernetes-Betriebskonzept (konzeptionell):** Beschreiben Sie fünf Bausteine:
 - Rollout-/Rollback-Strategie für kritische Services,
 - Skalierungsregel (Wenn-Dann),
 - Umgang mit Konfiguration und Secrets,
 - Monitoring/Alerting,
 - Ressourcenlimits.

Lernvideo: [Microservices — warum, wann und mit welchen Grenzen](#)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Aufgabe 7 – Fallstudie “InsureFast” – Teststrategie & Operating Model

L2 • Anwendung

~ 30 min

Fortsetzung: InsureFast

Bezug: Diese Aufgabe baut auf Aufgabe 6 auf. Sie setzen die Fallstudie “InsureFast” fort und ergänzen die fehlenden Bausteine.

Arbeitsauftrag (Teil 2 von 2):

1. **Teststrategie (TDD-Ansatz):** Definieren Sie für InsureFast eine Teststrategie mit:

- Unit-Tests für Kernlogik (welche Logik primär?),
- Integrationstests für Service-APIs (welche Schnittstellen?),
- E2E-Tests nur für *zwei* Kernpfade (welche Pfade wählen Sie?).

Ergänzen Sie *zwei Quality Gates*, die jeder Build durchlaufen muss – und eine *minimale* E2E-Absicherung.

2. **Operating Model:** Skizzieren Sie:

- den Teamzuschnitt (Produktteams + Plattformteam + Security/Compliance),
- eine RACI-Aufteilung für Schnittstellenänderungen,
- das Incident-Handling (wer wird wann gerufen?),
- den Release-Freigabepfad.

3. **Zwei Entscheidungen unter Unsicherheit:**

- *Datenstrategie:* Big-Bang-Migration vs. DB-bleibt + Anti-Corruption-Layer – was wählen Sie und warum?
- *Release-Frequenz:* Wöchentlich für alle Services vs. gestuft (Kern wöchentlich/zweiwöchentlich) – welche Variante?

Definieren Sie zu jeder Entscheidung ein **Frühwarnsignal**.

Lernvideo: Microservices — warum, wann und mit welchen Grenzen

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Aufgabe 8 – Anti-Muster “Microservices-Spaghetti” erkennen und korrigieren

L2 • Anwendung

~ 25 min

Vier Beispiele aus dem Alltag

A – “Service-Wildwuchs”: 47 Services für einen Mittelständler mit 60 Entwicklern. Jeder Service hat einen eigenen Datenstand, dieselben Kundendaten existieren mehrfach.

B – “Verteilter Monolith”: Fünf Services, aber jede Änderung erfordert *koordinierte Releases* aller fünf – weil sie sich gegenseitig synchron aufrufen.

C – “Observability als Nachgedanke”: Nach jedem Incident dauert es im Schnitt zwei Stunden, bis überhaupt klar ist, welcher Service den Fehler ausgelöst hat. Logs liegen verstreut in fünf Tools.

D – “Wir vertrauen unseren Tests” ohne Rollback-Übung: Niemand hat in den letzten sechs Monaten einen Rollback in der Produktion gefahren. Im Incident-Fall wird improvisiert.

Arbeitsauftrag:

Für jedes der vier Beispiele (A, B, C, D):

1. Benennen Sie das primäre **Anti-Muster** (z. B. “Verteilter Monolith”, “Service-Wildwuchs”, “Observability-Schuld”, “Rollback-Atrophie”).
2. Beschreiben Sie in einem Satz, **welches Prinzip** dabei verletzt wird (Servicegrenze entlang Capabilities, lose Kopplung, Betriebsdisziplin, you-build-it-you-run-it).
3. Schlagen Sie eine **konkrete Korrektur** in 1 – 2 Sätzen vor.

Lernvideo: Microservices — warum, wann und mit welchen Grenzen

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Niveau L3 – Management & Entscheidungsarchitektur

Aufgabe 9 – Reliability- & Risk-Framework für eine wachsende Plattform

L3 • Management

~ 35 min

Ausgangslage: Plattform-Skalierung

Ihre Rolle: Head of Platform einer Organisation mit acht Produktteams. Drei Teams releasen wöchentlich, fünf releasen alle 4–6 Wochen.

Beobachtung: Die Change Failure Rate liegt bei 18 % (Soll: $\leq 10\%$). MTTR ist hoch (Median 2,5 h). Es gibt keine einheitlichen SLOs.

Zielkonflikt: Geschwindigkeit der Produktteams vs. Stabilität und Auditierbarkeit der Plattform.

Arbeitsauftrag:

1. **SLO-Set entwickeln:** Definieren Sie ein Minimal-SLO-Set (drei SLOs) für kritische Services – mit Schwellenwerten und Begründung (Verfügbarkeit, Latenz, Fehlerquote).
2. **Error Budgets (konzeptionell):** Erklären Sie, wie ein Error-Budget-Mechanismus zwischen Speed und Reliability vermittelt. Definieren Sie eine Regel, was passiert, wenn das Budget aufgebraucht ist (Release-Stop? Plattform-Review?).
3. **Risk-Framework:** Erstellen Sie eine Liste von fünf typischen Risiken (z. B. *Servicegrenze falsch geschnitten*, *Secret im Code*, *Rollback nicht geübt*, *Observability-Lücken*, *Tooling-Lock-in*). Bewerten Sie jedes Risiko nach *Impact* und *Wahrscheinlichkeit* (jeweils 1–5) und benennen Sie pro Risiko eine präventive Maßnahme.
4. **Anti-Gaming-Mechanik:** Wie verhindern Sie, dass Teams ihre Change Failure Rate schönen (z. B. Fehler nicht als Change deklarieren, kleine Releases häufen, Postmortems vermeiden)? Definieren Sie zwei Gegenkopplungen.

Lernvideo: [Test-Driven Development — Qualität als Entscheidungsinstrument](#)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Aufgabe 10 – Governance leicht, aber wirksam

L3 • Management

~ 30 min

Diskussionsaufgabe.

Microservices und DevOps verlocken zu maximaler Teamautonomie. Ohne Leitplanken entstehen schnell Schatten-Architekturen, inkonsistente Schnittstellen und Audit-Findings. Schwergewichtige Architektur-Boards hingegen werden umgangen oder verzögern Releases. Hier ist die Management-Spannung.

Arbeitsauftrag:

1. **Architekturprinzipien (max. 8 Prinzipien):** Formulieren Sie acht Prinzipien, die in Ihrer Organisation als *verbindlich* gelten sollen (z. B. *Servicegrenzen folgen Capabilities*, *kein Secret im Code*, *jeder Service hat einen Owner*, *API-Versionierung verpflichtend*). Schreiben Sie jedes Prinzip in einem Satz und ohne Füllwörter.
2. **Schnittstellenstandards:** Definieren Sie zwei verbindliche Standards (z. B. *OpenAPI-Spezifikation für jede REST-API*, *Event-Schema-Registry für asynchrone Kommunikation*, *Idempotenz bei schreibenden Operationen*).
3. **Review-Regel:** Definieren Sie, wann ein *Architektur-Review* verpflichtend ist – und wann nicht. Faustregel: Review nur für *teure* oder *irreversible* Entscheidungen.
4. **Eskalationspfad:** Wer entscheidet, wenn Produktteam und Plattformteam sich uneinig sind? Skizzieren Sie den Pfad in vier Stufen.

Lernvideo: DevOps-Operating-Model — „you build it, you run it“ und Plattform-Teams

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the page.

.....

.....

.....

.....

Aufgabe 11 – Transferprojekt – Digitale Prozessarchitektur als Betriebs- und Entscheidungsarchitektur

L3 • Management

~ 45 min

Rolle und Auftrag

Ihre Rolle: Chief Architect / Head of Platform / Chief Digital Officer.

Auftrag: In **16 Wochen** eine skalierbare digitale Prozessplattform etablieren, die wöchentlich releasen kann *und* auditierbar bleibt (Microservices + Kubernetes + DevOps + Operating Model).

Restriktionen / Unsicherheit:

- Legacy-Landschaft, Lastprofile unbekannt,
- Budget **350 k €**,
- mehrere Teams mit unterschiedlichem Reifegrad,
- Security- und Audit-Druck,
- Zeitdruck durch Peak-Saison.

Arbeitsauftrag: Entwickeln Sie eine *Entscheidungsarchitektur* (keine Maßnahmenliste) für die Plattform. Erarbeiten Sie schriftlich:

1. **Top-10-Entscheidungsarchitektur:** Servicegrenzen, Datenhoheit, API-/Event-Standards, SLOs, Release-Policies, Security-Baselines, Observability-Standard, Incident-Governance, Ausnahmeprozess, Toolchain-Standardisierung. Für jede Entscheidung: Wer entscheidet? Welche Optionen?
2. **Operating Model & RACI:** Produktteams vs. Plattform vs. Security/Compliance. *Wer darf was ändern?* Eskalationspfad bei Zielkonflikten (Speed vs. Risk).
3. **Reliability- & Risk-Framework:** SLOs, Error Budgets, Change Failure Rate, MTTR. Definieren Sie eine *Anti-Gaming*-Kopplung.
4. **Governance leicht, aber wirksam:** max. acht Architekturprinzipien, Review nur für teure/irreversible Entscheidungen, Standard-Templates (CI/CD, Helm/Deployment – konzeptionell).
5. **Roadmap:** MVP (8 Wochen) → Scale (8 Wochen) inklusive *Messkonzept* (Outcomes, nicht Aktivität).
6. **Zwei Entscheidungen unter Unsicherheit (Pflicht):**
 - *Welche drei Services zuerst extrahieren?* Begründung + verworfene Alternativen + Frühwarnsignale.
 - *Welche Release-Policy für Kernprozesse?* Trade-off Speed/Safety + Guardrails.

Bewertungskriterien (Senior-Level): Entscheidungsklarheit · Anschlussfähigkeit an Strategie · Pragmatismus unter Restriktionen (16 Wochen, 350 k €, Reifegrad-Heterogenität) · Risikoreife (sichtbare, nicht verdrängte Risiken).

Lernvideo: Microservices — warum, wann und mit welchen Grenzen

.....

.....

.....

[illegible]

.....

Abschluss

Sie haben alle elf Aufgaben bearbeitet. Vergleichen Sie nun Ihre Antworten mit dem *Lösungsheft Tag 10*.

Was Sie jetzt können sollten

- Microservices nach Capability-Logik schneiden und Servicegrenzen begründen.
- Container/Kubernetes-Konzepte (Orchestrierung, Self-Healing, Rollback, Service-Discovery) zuordnen.
- Eine Testpyramide entwerfen und Quality Gates als Sicherheitsnetz formulieren.
- Ein Kubernetes-Betriebskonzept entwerfen (Rollout, Skalierung, Konfig/Secrets, Monitoring).
- Ein Operating Model mit RACI für Produktteams, Plattform und Security entwickeln.
- Ein Reliability-Framework (SLOs, Error Budgets, MTTR) inklusive Anti-Gaming-Kopplung entwerfen.
- Eine Architektur-Governance entwerfen, die leicht, aber wirksam ist – und Entscheidungen unter Unsicherheit dokumentieren.